



SECURITY REVIEW • PREPARED FOR

HoodLock

Trustless token locking and vesting on
Robinhood Chain

DATE

16.09.2026

COMMIT

880f9e69

REMEDIATION

880f9e69

Phase & process

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Telegram [@sbsecurity_audits](https://t.me/sbsecurity_audits).

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1 IMPACT

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2 LIKELIHOOD

- **High** - almost certain to happen, easy to perform, or highly incentivized.
- **Medium** - only conditionally possible, but still relatively likely.
- **Low** - requires specific state or little-to-no incentive.

02 EXECUTIVE SUMMARY

Key findings

TOTAL FINDINGS

0

RESOLVED

0

of 0 · 0%

HIGH SEVERITY FINDINGS

0

none found

SEVERITY DISTRIBUTION

☒ Fixed & verified ☒ Acknowledged

CRITICAL 0

HIGH 0

MEDIUM 0

LOW 0

INFORMATIONAL 0

GAS OPT. 0

CLIENT

HoodLock

REPOSITORY

Private

METHODS

Manual review

CONTRACTS IN SCOPE

src/RobinhoodLocker.sol

src/RobinhoodVesting.sol

TIMELINE · SEPTEMBER 2026

Audit · 2 days

Remediation · 1 day

KICK-OFF

15.09.2026

END OF AUDIT

16.09.2026

REMEDIATIONS START

16.09.2026

REMEDIATIONS END

16.09.2026

STARTING COMMIT HASH

880f9e69 Locker, 9f05ccfc Vesting

FINAL COMMIT HASH

880f9e69 Locker, 9f05ccfc Vesting

03 PROJECT SUMMARY

What HoodLock is

HoodLock is a pair of small, non-upgradeable contracts on Robinhood Chain. The Locker holds any ERC-20 or LP token until a chosen unlock time, and the Vesting contract releases tokens to a beneficiary on a linear curve with an optional cliff. Both charge a flat ETH fee at creation, and no admin function can move, unlock or shorten a lock or a schedule.

SECURITY POSTURE

BEFORE REMEDIATION

880F9E69

Custody of locked and vested tokens

Tokens leave the Locker only through the owner's withdraw after the unlock time, and the Vesting contract only through the beneficiary's claim on the linear curve. No admin function reaches them, and both paths update state before the transfer behind a reentrancy guard.

Fees and admin powers

The fee is a flat ETH amount charged once at creation. The Locker forwards it and refunds any excess, the Vesting contract accrues it for the collector to pull. The admin can only change the fee, the collector and the admin key.

Documented behaviour

Tokens without a return value cannot be locked, all schedules of one token share one balance, owner and beneficiary lists keep old ids after a transfer, and vesting dates may lie in the past. The team documents these, and they were confirmed as design choices rather than defects.

AFTER REMEDIATION

880F9E69

No issues were found, so there was nothing to fix. Both contracts are live on Robinhood Chain without an upgrade path, and the reviewed source is the code that runs there. The recommendations on the next page cover operational points around keys, token choice and integrations that sit outside the contracts themselves.

04 RECOMMENDATIONS

Beyond this audit

Measures that extend the protection of this review after launch, independent of any individual finding.

R-01

Keep the admin and fee collector on safe keys

The same admin and fee collector run both contracts. In the Locker the admin changes in one step and can set any fee, and the collector is paid inside lock, so it must always accept ETH or locking stops. Keep both keys on a multisig or hardware signer, and check that a new collector can receive ETH before you set it.

R-02

Publish token guidance for creators

Tell users which tokens work. The Locker rejects tokens whose transfer returns no value. The Vesting contract pools all schedules of one token, so rebasing, blacklisting or seizable tokens can leave the last claimers short. Both record fee-on-transfer tokens at the amount received, not the amount sent.

R-03

Read the current owner or beneficiary, not the index

locksByOwner and schedulesByBeneficiary keep an id after a transfer. Frontends and indexers should read the owner or beneficiary from the lock or schedule itself and follow the transfer events instead of trusting the lists.

R-04

Show what the transaction will do before the user signs

A lock can unlock seconds after creation, and a vesting schedule can start in the past and be partly claimable right away. Show the unlock time, or the amount claimable at creation, so a wrong date is caught before the tokens are deposited.

Core contracts

CONTRACT

ROLE

`src/RobinhoodLocker.sol`

One vault for many locks. Anyone can lock an ERC-20 or LP token until a chosen unlock time, and only the lock owner can withdraw after that time, extend it to a later date or hand the lock to another wallet. The flat ETH fee is paid to the collector inside lock and any excess is refunded. Live at `0xd0f7d8c6e9f6d80c297bebe4f7fd1b9c8125c32f`.

`src/RobinhoodVesting.sol`

Creates schedules through `create` and `createMany`, holds the deposited tokens, pays out claims on the linear curve and lets a beneficiary move a schedule to a new wallet. The creation fee accrues in the contract and the collector pulls it. Live at `0x910e19bcC4bce46999994Ed7297E0Fc4431ec72E`.

Privileged roles

Admin.

Held by 0x79c1230cAb12d53D040f5FE1F5279e1A481CCeA2 for both contracts. Sets the creation fee, changes the fee collector and replaces the admin. In the Locker the fee has no cap and the admin changes in one step; in the Vesting contract the fee is capped at 0.05 ETH and the transfer is two-step. Cannot touch any lock or schedule.

Fee collector.

Held by 0xd59948E49B1B56784626bDe2d6D1712eD24a6BBc for both contracts. Receives the Locker fee at lock time, so it must accept ETH, and pulls the accrued Vesting fees through withdrawFees.

Lock owner.

Set per lock to whoever created it. The only address that can withdraw, extend or transfer that lock.

Beneficiary.

Set per schedule by the creator. The only address that can claim from that schedule or transfer it to another wallet. The creator has no rights over a schedule once it is created.

What we found

ID	TITLE	SEVERITY	STATUS
----	-------	----------	--------



AUDIT COMPLETE

Stay secure.

Thank you for choosing SBSecurity.

WEBSITE

sbsecurity.net

TELEGRAM

[@sbsecurity_audits](https://t.me/sbsecurity_audits)

REPORT

HoodLock · September 2026